PROJECT REPORT

SAFE-Drive

A Research Toolkit for AI-Assisted Road Safety Monitoring

UPDATED REPORT

Project focus
Traffic-video analysis, vehicle tracking, calibrated trajectory measurements, near-miss screening and human review.

Current development snapshot
Four implementation phases reported complete; premium landing-page redesign and final GitHub push confirmation remain outstanding in the supplied log.

Repository
github.com/bhuvzzs24-spec/SAFE-Drive

Prepared from the prior project report and the latest supplied Codex development log. Implementation and test claims are attributed to that log unless independently verified.

Contents

Abstract

1. Introduction

2. Problem Statement and Objectives

3. Existing and Proposed System

4. System Architecture

5. Tools and Technologies

6. AI Models and Algorithms

7. Backend and Data Management

8. Frontend and User Interface

9. Security and Deployment Readiness

10. Testing and Validation

11. Current Results and Discussion

12. Limitations

13. Future Work

14. Conclusion

References

Status note: this revision incorporates the latest reported implementation and test status. It does not claim that real-video inference, production WSGI operation, public deployment, or the final GitHub push has been verified.

Abstract

SAFE-Drive is a research toolkit for AI-assisted road-safety monitoring. It brings together a Flask web application, traffic-video processing, YOLO-based object-detection integration, DeepSORT multi-object tracking, camera-specific road-plane calibration, trajectory-based speed estimation, closest-approach and time-to-collision screening, configurable risk indicators, SQLite persistence, and human review.

The latest development log records substantial work across four implementation phases: backend reliability, dashboard usability, computer-vision integration, and security/deployment preparation. It reports 83 passing Python tests and 8 passing frontend tests after the security phase. These results indicate tested software behaviour, but they do not establish real-world detection accuracy or end-to-end video performance.

At the time of the supplied log, compatible detector weights and representative test footage were not available, the production WSGI smoke test remained unverified, and the final GitHub push was not confirmed. The premium, cinematic landing-page redesign is a separate remaining task. SAFE-Drive should therefore be described as an improved research prototype, not as a fully validated production road-safety system.

1. Introduction

Traffic-video review is time-consuming when operators must manually locate road-user interactions across long recordings. Computer vision can help organize this work by detecting road users, tracking their movement, and identifying trajectory patterns that merit further review.

SAFE-Drive provides a modular workflow for analysing traffic video and retaining reviewable results. It is intended to assist research and human review, not to make legal determinations, issue fines, establish fault, or guarantee that an accident will occur.

1.1 Potential applications

- Traffic-video analysis and road-user trajectory studies.
- Research into calibrated speed estimation.
- Screening potential near-miss interactions for human review.
- Exploration of intelligent transportation systems and video analytics.

2. Problem Statement and Objectives

Manual traffic-footage review requires substantial time, while speed estimates derived only from image pixels can be unreliable when perspective and road geometry are ignored. A useful research toolkit should combine video handling, trajectory tracking, valid calibration, interpretable screening signals, persistent records, and a review interface.

2.1 Objectives

- Provide a dashboard for uploaded video and supported configured camera streams.
- Integrate road-user detection with multi-object tracking when compatible dependencies and model weights are available.
- Estimate speed from timestamped trajectories only when camera-specific calibration is valid.
- Screen interactions using closest-approach and time-to-collision indicators.
- Generate configurable risk signals with explicit limitations.

- Persist sessions, tracks, events, and human-review decisions.
- Support reproducible testing and future evaluation against independently labelled data.

3. Existing and Proposed System

Conventional workflows often separate CCTV observation, speed measurement, and event review. SAFE-Drive proposes a single research interface connecting video input, perception, geometric analysis, screening, storage, and human review.

The application is modular: optional vision dependencies and compatible weights are required for actual inference; calibration is specific to a camera view; speed estimates remain unavailable when valid calibration is absent; and screening events are indicators for review rather than proof of a collision.

4. System Architecture

The architecture is organized into five conceptual layers: input, perception, analysis, persistence, and application/review. The diagram below summarizes the intended flow.

INPUT	Uploaded traffic video	Configured RTSP/RTSPS stream
PERCEPTION	Frame acquisition	YOLO detection → DeepSORT tracking
ANALYSIS	Trajectory building	Calibration → speed → closest approach/TTC → risk rules
PERSISTENCE	SQLite sessions/tracks/events/reviews	Local video files and JSONL event records
APPLICATION	Flask dashboard	Human review: verify / reject

The latest supplied Codex log reports improvements to event persistence, timestamp handling, detector loading validation, tracker input validation, and job supervision. The report does not independently inspect the current repository state; these implementation details are summarized from that log.

5. Tools and Technologies

Technology	Role	Status / notes
Python 3.10+	Application and analysis modules	Core runtime
Flask	Web application and backend routes	Core
HTML, CSS, JavaScript	Dashboard and browser interactions	Existing interface; redesign remains planned
SQLite	Sessions, tracks, events, review decisions	Local persistence
NumPy	Numerical calculations and geometry	Core numerical dependency
OpenCV	Video frames and camera streams	Optional vision dependency; native import/playback check was unresolved in the log
Ultralytics YOLO	Object-detection integration	Compatible local weights required; not bundled in the project at audit time

Technology	Role	Status / notes
DeepSORT	Multi-object tracking	Tracking integration; real footage behaviour not validated
Gunicorn	Production WSGI serving	Installed in the environment according to the log; final HTTP smoke test unverified
Git and GitHub	Version control and source hosting	Final push not confirmed in supplied log

Optional vision dependencies and model files must be configured for real inference. Camera credentials, private footage, secrets, local databases, and generated evidence should not be committed to the public repository.

6. AI Models and Algorithms

6.1 YOLO-based object detection

YOLO is integrated as the object detector for supported road-user classes. Actual inference requires compatible, explicitly supplied model weights. A general-purpose detector does not automatically detect helmet violations.

6.2 DeepSORT tracking

DeepSORT associates detections across frames and assigns session-local track identifiers. Tracking quality depends on detections, scene conditions, frame timing, and tracker configuration; real-world identity consistency remains unverified.

6.3 Road-plane homography

A homography maps image points to measured road-plane coordinates. It requires at least four well-distributed, non-collinear point correspondences and is specific to a camera view.

6.4 Speed estimation

Speed is calculated from calibrated positions and timestamps. The result depends on calibration quality, trajectory quality, and timing. Without valid calibration, speed should be reported as unavailable rather than inferred from pixels alone.

6.5 Near-miss screening and risk scoring

Closest-approach and time-to-collision indicators screen trajectory pairs for interactions that may deserve review. Configurable risk scores are research indicators, not validated accident probabilities, legal thresholds, or evidence that a collision occurred.

6.6 Helmet and accident detection

Helmet-state inference remains experimental and requires compatible dedicated weights and labelled evaluation data. Accident detection is not implemented and must not be claimed as a current capability.

7. Backend and Data Management

The Flask application coordinates configuration, sessions, analysis jobs, persistence, and dashboard responses. Reusable analysis modules live under the project’s source package. SQLite stores sessions, track summaries, events, and review decisions; uploaded clips and generated outputs are stored as files.

The supplied implementation log reports changes addressing event serialization, input validation, processing limits, access controls, job recovery, and safer camera configuration. It also reports a read-only public-demo mode and operator access controls. These changes should be rechecked against the current repository and deployment

configuration before any public exposure.

A deployment must retain persistent storage for the database and evidence files. Secrets must be supplied through environment configuration, not committed. The production WSGI health check and complete hosted deployment were not confirmed by the supplied log.

8. Frontend and User Interface

The existing dashboard was improved during the earlier phases. The supplied log reports nine browser views, unified styling, light/dark themes, mobile navigation, searchable history, track/event inspection, event filters, calibration guidance, improved processing/error states, and a fix for local video preview URLs.

The same log reports that all nine views were checked at 320 px and 1280 px without horizontal overflow. This is a reported check from the development log, not an independent visual review of the current live site.

8.1 Remaining UI objective

The requested premium redesign is a separate task: a cinematic landing hero, scroll-driven product storytelling, refined typography, restrained navy/green/teal colour, subtle scroll reveals, polished interactions, and consistent styling across both the landing page and operational dashboard. The redesign should preserve real backend actions and must not substitute mock data for real analysis.

8.2 Intended visual system

- Primary: deep navy blue for structure and contrast.
- Accent: teal/green for primary actions and positive status.
- Neutral surfaces: off-white or pale blue-green for readable content areas.
- Motion: subtle reveal and hover effects with reduced-motion support.
- Operational priority: video analysis and review remain easy to access without navigating through the full landing page.

9. Security and Deployment Readiness

The audit initially identified risks around arbitrary camera destinations, unbounded job submissions, unauthenticated access to saved evidence, and local-only job state. The subsequent implementation log reports public-demo restrictions, role-based operator access, server-configured camera sources, bounded processing, subprocess job supervision, restart-recovery work, environment-based configuration, and deployment documentation.

The log reports 83 passing Python tests and 8 passing frontend tests after the security work. It also states that elevated approval review blocked the final WSGI HTTP probe and restart command, so the production WSGI smoke test remained unverified.

No hosted service was created, and public deployment was not performed. Hosting suitability, persistence configuration, access-control configuration, and real deployment behaviour require separate verification.

10. Testing and Validation

10.1 Automated tests reported

Check	Reported outcome	Interpretation
Python test suite after Phase 4	83 passed; no failures or skips	Software tests reported by Codex; not an independent rerun here
Frontend tests	8 passed	Reported by Codex
Earlier Phase 2 Python tests	33 passed	Earlier checkpoint; superseded by the later 83-test report
Responsive view check	320 px and 1280 px; no horizontal overflow	Reported check for nine views
Production WSGI HTTP smoke test	Unverified	Probe did not run because approval review blocked the command
Real-video inference	Unverified	Compatible detector weights and suitable footage were unavailable
Live camera operation	Unverified	No real camera operation test was completed

10.2 Recommended real-world evaluation

Area	Recommended measure
Detection	Precision, recall, mAP at a declared IoU threshold
Tracking	HOTA or IDF1 and identity-switch review
Speed estimation	MAE/RMSE against independently measured speed
Near-miss screening	Precision/recall against manually annotated ground truth
Calibration	Error against independent road distances or surveyed points
Performance	Processing latency and frames per second on documented hardware
Robustness	Different camera views, lighting, weather, and traffic densities

Software tests validate the behaviours they cover. They do not establish detector accuracy, tracking quality, speed accuracy, near-miss recall, or safety benefit. Those claims require representative labelled footage and independent measurements.

11. Current Results and Discussion

The development log indicates meaningful engineering progress: integration boundaries were tightened, security controls were added, UI usability was improved, and the reported automated suite expanded to 83 Python tests plus 8 frontend tests.

The project is not yet fully validated as an end-to-end AI demonstration. No compatible detector checkpoint or representative test video was available in the audit environment, and the native OpenCV import stalled during a playback-fixture check. Therefore, this report does not claim measured detection accuracy, tracking accuracy, speed accuracy, near-miss performance, or processing throughput.

The supplied log does not confirm that the final staged changes were committed and pushed to GitHub. Repository synchronization must be checked with Git status, remote comparison, commit history, and a successful push confirmation.

12. Limitations

- End-to-end inference on representative labelled traffic footage has not been verified.
- Compatible YOLO weights and a suitable demo video are required for a real detection demonstration.
- Real-world speed estimation requires camera-specific calibration and independent validation.
- Helmet inference is experimental and requires compatible weights; accident detection is not implemented.
- Risk scores and near-miss indicators are screening aids, not validated probabilities or legal determinations.
- The final production WSGI HTTP smoke test was not completed in the supplied log.
- The final GitHub push was not confirmed in the supplied log.
- The premium scroll-driven landing-page redesign remains a separate planned task.
- Public deployment and 24/7 availability have not been established.

13. Future Work

- Confirm all intended changes are safely committed and pushed to the correct GitHub repository.
- Complete the premium landing page and apply the same visual system to all existing application views.
- Supply a compatible local detector checkpoint and licensed representative traffic video.
- Run and document a reproducible end-to-end video-processing test.
- Validate detection, tracking, speed, calibration and near-miss screening against independent ground truth.
- Complete the production WSGI smoke test and validate deployment configuration.
- Review persistence, backups, retention, privacy and access controls before public exposure.
- Evaluate helmet-state inference only with suitable labelled data and compatible trained weights.

14. Conclusion

SAFE-Drive is a modular research toolkit combining a Flask dashboard, optional YOLO detection, DeepSORT tracking, camera-specific geometry, trajectory-based speed estimation, closest-approach/TTC screening, configurable risk signals, SQLite persistence, and human review.

The latest development log reports substantial improvements across four implementation phases and 91 passing automated tests in total across Python and frontend suites (83 Python plus 8 frontend). These results are encouraging but remain software-test results, not proof of end-to-end AI accuracy.

The project should be presented as an improved research prototype with explicit prerequisites and limitations. The next practical milestones are to confirm the GitHub push, finish the premium UI, and perform a real-video smoke test with compatible weights before describing the full AI pipeline as demonstrated.

References

[1] SAFE-Drive, “SAFE-Drive: A research toolkit for AI-assisted road safety monitoring,” GitHub repository. Available: https://github.com/bhuvzzs24-spec/SAFE-Drive

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” CVPR, 2016, pp. 779–788.

[3] Ultralytics, “YOLO documentation.” Available: https://docs.ultralytics.com/

[4] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” ICIP, 2017, pp. 3645–3649.

[5] R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed. Cambridge University Press, 2004.

[6] OpenCV, “Open source computer vision library documentation.” Available: https://docs.opencv.org/

[7] Flask, “Flask web framework documentation.” Available: https://flask.palletsprojects.com/

[8] NumPy, “NumPy documentation.” Available: https://numpy.org/doc/

[9] SQLite, “SQLite documentation.” Available: https://www.sqlite.org/docs.html